

Informe de Justificación Técnica de Diseño Software

Análisis de Mapeo Sintáctico, Patrones GRASP y Principios SOLID

Proyecto: The Shiunji Family Children **Autor:** Valentino Pippo
Dominio: Modelado Orientado a Objetos (UML & Swing) **Materia:** Análisis y Diseño de Software / POO

1. Resumen Ejecutivo y Objetivos

El presente documento constituye el informe técnico de auditoría y arquitectura de software para el sistema "The Shiunji Family Children". El objetivo fundamental es analizar la concordancia técnica entre el modelo estático/dinámico (Diagrama de Clases UML y Diagrama de Secuencia) y la implementación concreta en el lenguaje de programación Java (Swing GUI), justificando de manera rigurosa la aplicación de los principios de diseño **SOLID** y los patrones **GRASP** (General Responsibility Assignment Software Patterns).

2. Análisis Comparativo: Artefactos UML vs. Código Implementado

Observación Arquitectónica: Divergencia Nominal de Actores y Nombres

Al auditar el Diagrama UML frente al código fuente `Main.java`, se observa una adaptación en los nombres del personaje principal y su jerarquía de hermanos:

- Diagrama UML:** El protagonista principal es denominado `Shiunji_Taisei` / `Taisei` y interactúa con 5 hermanas (`Eldest_Sister`, `Segundo_Hermano`, `Cuarta_Hermana`, `Quinta_Hermana`, `Sexta_Hermana`).
- Código Java:** Modela de forma completa a los 7 hermanos con sus nombres canónicos: `Arata` (protagonista equivalente a Taisei), `Banri`, `Seiha`, `Ouka`, `Minami`, `Shion` y `Kotono`, además de la clase `Padre` (`Kaname Shiunji`).

2.1 Mapeo de Entidades y Estructura Clase-Clase

Entidad UML	Clase Java en Código	Responsabilidad y Comportamiento Implementado
<code>Persona</code> (N/A en UML implícito)	<code>static abstract class Persona</code>	Clase base abstracta que encapsula <code>nombre</code> y <code>edad</code> con su método polimórfico <code>presentarse()</code> .
<code>Eldest_Sister</code> / <code>Hermano</code>	<code>static class Hermano extends Persona</code>	Especialización que añade el atributo <code>personalidad</code> e interacciones dinámicas.
<code>Taisei</code> / <code>Shiunji_Taisei</code>	<code>static class Arata extends Hermano</code>	Encarna el rol protector. Provee el método específico <code>cuidarHermanos()</code> .
<code>Padre</code> / <code>Madre</code>	<code>static class Padre extends Persona</code>	Provee lógica de soporte y revela el secreto familiar mediante <code>revelarSecreto()</code> .

Entidad UML	Clase Java en Código	Responsabilidad y Comportamiento Implementado
FamiliaShiunji	<code>static class FamiliaShiunji</code>	Agregador de alto nivel que gestiona el arreglo de <code>Persona[] miembros</code> y la referencia a <code>Casa</code> .
Casa	<code>static class Casa</code>	Abstrae la infraestructura física (<code>direccion</code> , <code>habitaciones</code>) con <code>recibirFamilia()</code> .
Evento	<code>static class Evento</code>	Representa los acontecimientos y su impacto relacional mediante <code>ocurrir()</code> .

3. Justificación Rigurosa de Principios SOLID

S — Single Responsibility Principle (SRP)

Principio: Una clase debe tener una sola razón para cambiar.

Justificación en el Código: Cada clase del dominio posee un propósito conceptual único y delimitado. Por ejemplo, `Casa` únicamente mantiene los datos estructurales del hogar y gestiona la albergabilidad de la familia. La lógica de presentación gráfica Swing se encuentra totalmente aislada en las clases visuales (`BackgroundPanel` y los botones de `Main`), garantizando que las entidades del modelo (`Hermano` , `Evento`) no dependan de APIs de renderizado.

O — Open/Closed Principle (OCP)

Principio: Las entidades de software deben estar abiertas a la extensión pero cerradas a la modificación.

Justificación en el Código: La jerarquía `Persona -> Hermano -> [Arata, Banri, Seiha, ...]` demuestra una extensible jerarquía de herencia. Para agregar un nuevo integrante a la familia Shiunji, basta con extender la clase abstracta `Persona` o la subclase `Hermano` implementando sus comportamientos distintivos, sin alterar la estructura interna de la clase contenedora `FamiliaShiunji`, la cual opera sobre arreglos polimórficos de tipo `Persona[]`.

L — Liskov Substitution Principle (LSP)

Principio: Los objetos de una subclase deben poder sustituir a los objetos de la superclase sin alterar la coherencia del programa.

Justificación en el Código: En el método `mostrarFamilia()`, se crea una colección de tipo `Persona[] miembros` que alberga instancias heterogéneas (`Arata`, `Banri`, `Padre`, etc.). La clase `FamiliaShiunji` invoca transparentemente `getNombre()` y `getEdad()` sobre cada elemento sin requerir verificaciones explícitas de tipo o *casting* forzado, respetando el contrato de la superclase `Persona`.

I — Interface Segregation Principle (ISP)

Principio: Los clientes no deben ser obligados a depender de interfaces que no utilizan.

Justificación en el Código: Las clases están especializadas con métodos precisos. La subclase `Arata` define únicamente `cuidarHermanos()`, `Banri` define `estudiar()` y `Seiha` define `analizar()`. Ninguna subclase sobrecarga o implementa métodos vacíos o irrelevantes para su comportamiento específico.

D — Dependency Inversion Principle (DIP)

Principio: Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

Justificación en el Código: La clase colectora `FamiliaShiunji` depende de la abstracción `Persona[]` en su constructor en lugar de acoplarse directamente a las clases concretas de cada hermano. Esto permite inyectar cualquier combinación de subclases de `Persona` mediante inversión de dependencias simple a nivel de datos.

4. Justificación Rigurosa de Patrones GRASP

GRASP Experto en Información (Information Expert)

Se asigna la responsabilidad a la clase que posee la información necesaria para realizarla.

Ejemplo en código: La clase `FamiliaShiunji` es la Experta en Información para formatear la lista de integrantes en el método `mostrarMiembros()`, ya que posee la referencia directa al arreglo `miembros`. Del mismo modo, `Casa` responde sobre sus propias capacidades físicas con `recibirFamilia()`.

GRASP Creador (Creator)

Una clase debe ser responsable de crear instancias de otra si las contiene o las utiliza estrechamente.

Ejemplo en código: La clase `Main` actúa como la creadora del contexto inicial, instanciando los objetos `Arata`, `Banri`, `Casa` y ensamblando la `FamiliaShiunji`.

GRASP Controlador (Controller)

Asigna la responsabilidad de recibir o manejar un evento del sistema a una clase no de la interfaz de usuario.

Ejemplo en código: Los controladores de eventos en Swing (`ActionListener`) mapean las acciones del usuario (click en botones como "Diagrama de Secuencia" o "Ver Familia") e invocan los métodos del dominio encargados de la simulación y la lógica interna.

GRASP Alta Cohesión y Bajo Acoplamiento (High Cohesion / Low Coupling)

Las clases presentan responsabilidades altamente focalizadas y con dependencias mínimas necesarias.

Ejemplo en código: La clase `BackgroundPanel` se encarga de manera altamente coherente del escalado,

dibujado y renderizado del fondo gráfico con gradientes de reserva, manteniendo un acoplamiento mínimo respecto al resto de la lógica de negocio.

GRASP Polimorfismo (Polymorphism)

Maneja alternativas de comportamiento basadas en el tipo de objeto.

Ejemplo en código: El método `presentarse()` definido en la superclase abstracta `Persona` es heredado por todas las variantes de miembros de la familia, permitiendo un tratamiento polimórfico unificado al recorrer la colección de miembros.

5. Conclusión y Recomendaciones de Refactorización

El diseño implementado en Java demuestra un cumplimiento sólido de las buenas prácticas de la programación orientada a objetos. Para alcanzar un nivel de excelencia arquitectónica óptimo en futuras versiones del proyecto, se recomiendan las siguientes mejoras:

- **Desacoplamiento Estricto de GUI:** Separar la clase contenedora `Main` en módulos independientes (paquete `model` para el dominio y `view` para los componentes Swing).
- **Sincronización Semántica UML:** Homologar la nomenclatura de los personajes entre los diagramas visuales y el código Java (por ejemplo, renombrar formalmente la clase `Arata` a `Taisei` o actualizar el diagrama de clases para reflejar el elenco completo).